AI Team Formation Tool: An Intelligent Constraint-Based System for Fair and Balanced Team Allocation Using Artificial Intelligence

Deepashri Murali
dept. of Computer science and Engineering
Rajalakshmi Engineering College
Chennai, India
deepashrimurali.2024.cse@rajalakshmi.edu.in

Bushra Banu KM
dept. of Computer science and Engineering
Rajalakshmi Engineering College
Chennai, India
bushrabanu.km.2024.cse@rajalakshmi.edu.in

Abstract— Forming effective project teams is a recurring challenge in academic and professional settings. Teams that are assembled manually or at random are often unbalanced in size and skills, may lack members who can take critical roles such as team lead or designer, and usually ignore practical constraints such as who should or should not work together and where members are located. This paper presents the **AI Team Formation Tool**, an intelligent decision-support system that combines several classical artificial intelligence techniques in a single pipeline. A forward-chaining rule engine infers the roles each participant is eligible for from skills, attendance, portfolio submission, and leadership experience, and records a complete reasoning trace. The inferred roles are passed to a constraint satisfaction problem (CSP) solver that uses union-find grouping, backtracking search with symmetry breaking, and incremental constraint checking to form balanced teams under size, togetherness, separation, and critical-skill constraints. When perfect coverage is impossible, the solver relaxes the requirement and reports exactly which skills or roles are missing instead of failing silently. Fairness metrics quantify size and skill balance, and an A* search over a campus graph recommends a meeting point for every team. On a 12-participant test cohort, the solver formed three balanced teams (sizes 5, 4 and 3) in under 4 ms whenever full coverage was feasible, and A* returned the same optimal walking distances as Dijkstra's algorithm with about 49% fewer node expansions. The tool is implemented as an interactive Streamlit application.

Keywords— Team Formation, Artificial Intelligence, Constraint Satisfaction Problem, Backtracking Search, Forward Chaining, A* Search, Fairness, Role Eligibility, Decision Support System.

I INTRODUCTION

Collaborative projects are a central part of modern education and the workplace. The success of a project depends not only on the abilities of individual members but also on how those members are combined. Factors such as technical skills, reliability, leadership experience, prior work, availability, and physical location influence which teams are likely to function well.

For example, a team without a member who can write code or design the interface may be unable to complete its deliverables, while one team holding most of the strong members and another holding none creates an unfair distribution of ability. Similarly, instructors and organizers often need to keep certain people together, keep other pairs apart, and ensure that every team has at least one person who is eligible to lead it.

Despite the availability of project-management and collaboration tools, team formation is still commonly done through random assignment, self-selection, or manual balancing in spreadsheets. These approaches are time-consuming, difficult to scale, and hard to justify when participants ask why they were placed in a particular team. They also rarely check eligibility rules or report what went wrong when constraints conflict.

Another limitation is the separation between related sub-problems. Deciding who is qualified for a role, dividing participants into teams, measuring whether the result is fair, and arranging where each team can meet are usually handled in different tools or by hand, so the output of one step does not automatically become the input of the next.

Artificial intelligence offers well-established techniques for these problems. Logical inference can derive eligibility from explicit rules and explain its conclusions, constraint satisfaction can search for assignments that respect hard requirements, and graph search can find efficient paths on a map. When these techniques are combined, team formation can be automated while the reasoning remains transparent.

To address these challenges, this paper proposes the **AI Team Formation Tool**, an AI-powered system for fair and balanced team allocation. The proposed system analyzes each participant's skills, attendance, portfolio status, leadership experience, and campus location, derives role eligibility using forward chaining, and forms teams using a backtracking CSP solver. It also provides fairness metrics and an A*-based meeting-point recommendation for each team.

The proposed system aims to integrate **rule-based eligibility inference, constraint-based team formation, fairness evaluation, and meeting-point search** into a unified platform. Rather than replacing human judgment, the tool is designed as a decision-support system that explains its inferences, reports unmet requirements, and leaves the final decision with the instructor or organizer.

II. LITERATURE SURVEY

Research on team formation, collaborative learning, and intelligent decision support has expanded considerably with the development of heuristic search, optimization, and machine learning techniques. Automated team formation is

increasingly explored as an alternative to random or self-selected grouping, because it can balance teams according to explicit criteria.

Traditional team formation in classrooms generally relies on instructor judgment, random assignment, or student self-selection. Such approaches are simple but may not sufficiently account for differences in skills, experience, availability, and interpersonal constraints. Manual balancing also requires considerable effort and must be repeated whenever the participant list or the requirements change.

Optimization-based and heuristic approaches, including clustering, genetic algorithms, and greedy assignment, have been explored for grouping participants by skill and performance. These methods can produce balanced groups, but they often treat requirements as weighted objectives. As a result, hard constraints such as keeping two people apart may be violated, and the reason for a particular assignment is difficult to explain.

Constraint satisfaction and constraint programming provide a natural formulation of team formation. Participants are variables, teams are values, and requirements such as size limits and forbidden pairs are constraints. Backtracking search with constraint checking, symmetry breaking, and variable-ordering heuristics can solve such problems efficiently for classroom-sized groups, and relaxation strategies can handle over-constrained cases.

Rule-based expert systems and logical inference have long been used to encode eligibility and decision rules. Forward chaining repeatedly applies rules to known facts until no new facts can be derived, and the sequence of fired rules can be presented as an explanation. Explainable reasoning is particularly valuable when decisions affect people, such as who is eligible to lead a team.

Graph search algorithms such as Dijkstra's algorithm and A* are widely used for route planning. A* uses a heuristic estimate of the remaining distance to focus the search, and with an admissible heuristic it returns optimal paths while expanding fewer nodes. These algorithms can be applied to practical coordination problems such as choosing a convenient meeting location.

However, existing approaches still present several limitations. Many tools focus only on the grouping step, while others concentrate only on scheduling or communication. Some provide fixed or random assignments with little explanation, and few report precisely which requirement could not be met when constraints conflict. Furthermore, role eligibility, team formation, fairness evaluation, and logistics are rarely connected in one workflow.

Therefore, there is a need for an integrated system that combines **rule-based eligibility reasoning, constraint-based team formation, fairness measurement, and search-based coordination**. The AI Team Formation Tool addresses this gap by feeding the output of each stage into the next within a single, explainable framework.

III. METHODOLOGY

The proposed **AI Team Formation Tool** forms teams using explicit participant data and classical AI techniques. The overall workflow consists of six major phases:

1. Participant Profile Collection
2. Role Eligibility Inference (Forward Chaining)
3. Team Formation as a Constraint Satisfaction Problem
4. Critical-Skill Coverage and Constraint Relaxation
5. Fairness Evaluation and Meeting-Point Search
6. Interactive Presentation and Explanation

The overall workflow of the proposed system is illustrated in Figure 1.

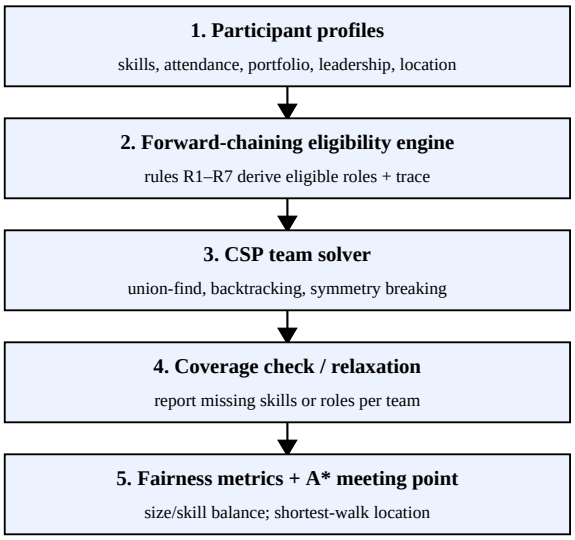

Fig. 1. Workflow of the proposed AI Team Formation Tool.

A. Participant Profile Collection

The first stage collects the information required for team formation. Each participant is entered in an editable table, and the system builds an individual profile from it.

The profile includes:

- Name
- Skills (for example Python, Design, Writing, PM, ML, Testing, DevOps, UX)
- Attendance percentage
- Portfolio submission status
- Leadership experience
- Current campus location

Attendance is converted into a level: 85% or above is treated as high, 60% to 84% as medium, and below 60% as low. The profile is then converted into a set of facts, such as *has_skill:Python*, *attendance:high*, and *leadership_experience*, for the reasoning engine.

B. Role Eligibility Inference Using Forward Chaining

The eligibility engine applies a small rule base repeatedly to the known facts until no new fact can be derived (a fixed point). The rules are:

- R1: Python skill and high attendance imply Developer eligibility
- R2: Design skill and a submitted portfolio imply Designer eligibility

- R3: Leadership experience and high attendance imply Team Lead eligibility
- R4: Writing skill and medium-or-high attendance imply Writer eligibility
- R5, R6: High or medium attendance imply medium-or-high attendance
- R7: PM skill and Team Lead eligibility imply Project Manager eligibility

Because R7 depends on a conclusion of R3, the engine performs multi-step inference. Every fired rule is recorded with the facts it used, so that the reasoning trace can be shown to the user. The derived roles are added to the participant's skill set as *Role:<name>* tokens so that they can be used directly as team constraints.

C. Team Formation as a Constraint Satisfaction Problem

Team formation is modeled as a CSP. Participants are the variables and team identifiers are the domain. The solver enforces:

- Every team size lies within the minimum and maximum limits
- Must-not-be-together pairs are never placed in the same team
- Must-be-together pairs are always placed in the same team
- Critical skills or roles appear in every team (requirement that may be relaxed)

Participants who must be together are merged into a single unit using union-find, so the constraint is satisfied by construction. Units are ordered by the number of separation conflicts and by size, so the most constrained units are placed first, in the spirit of the most-constrained-variable heuristic.

The search is a backtracking algorithm with two optimizations. First, since teams are interchangeable, a new team index is opened only one above the highest index already used (symmetry breaking), which removes many equivalent branches. Second, when a team becomes full, its skill coverage is checked immediately (incremental constraint propagation), allowing the search to prune a branch early. A node limit of 20,000 prevents unbounded search.

D. Critical-Skill Coverage and Constraint Relaxation

Requiring every team to hold every critical skill can be impossible, for example when only one participant is eligible for a role but several teams need it. In such cases the solver does not simply fail. It relaxes the coverage requirement, solves again using only the hard constraints, and returns, for each team, the list of critical skills or roles that could not be covered. This gives the organizer an actionable diagnosis instead of an empty result.

E. Fairness Evaluation and Meeting-Point Search

After the teams are formed, fairness metrics are computed: the team sizes, the average size and variance of sizes, the number of unique skill and role tokens per team, and the variance of that count. These values are displayed as bar charts.

The tool then recommends a meeting point for each team. The campus is modeled as a weighted graph of 12 locations connected by 19 walking paths, with edge weights equal to Euclidean distance. For every candidate location, shortest-path distances from all team members are computed using A* with a straight-line-distance heuristic or Dijkstra's algorithm. The tool can minimize either the total walking distance (efficiency) or the longest individual walk (fairness to the worst-off member).

F. Interactive Presentation and Explanation

The system is implemented as a Streamlit web application with Plotly charts. Users edit participant data, choose the number of teams, size limits, critical skills, and pairing constraints, and obtain the formed teams, solver statistics (nodes visited, backtracks, time), fairness charts, and meeting points. A reasoning-trace viewer shows how each role was derived, and a query interface reports whether a chosen target role can be established from a participant's recorded attributes.

IV. RESULT AND DISCUSSION

The proposed AI Team Formation Tool was evaluated on a built-in cohort of 12 participants with varied skills, attendance, portfolio status, leadership experience, and locations, and on randomly generated cohorts. All experiments were run with the implemented modules; times were measured on a standard laptop-class CPU and should be read as indicative.

A. Role Eligibility Inference

Nine of the 12 participants were found eligible for at least one role, while three (Frank, Grace and Ivan) were eligible for none because they lacked the required combination of skill, attendance, portfolio, or leadership evidence. Table I summarizes the result.

TABLE I. ROLE ELIGIBILITY DERIVED BY FORWARD CHAINING

Role	Eligible participants	Count
Developer	Alice, Carol, Judy, Liam	4
Designer	Bob, Dave, Heidi, Karl	4
Writer	Carol, Eve, Heidi	3
Team Lead	Alice, Dave, Liam	3
Project Manager	Dave	1

The cascade is visible in Dave's trace. R2 gives Designer, R3 gives Team Lead, R5 gives medium-or-high attendance, and R7 then derives Project Manager from the Team Lead conclusion, so four rules fire in total. Dave is the only participant eligible as Project Manager, which becomes important in Section C.

B. Team Formation and Constraint Satisfaction

Five scenarios were solved with 3 teams and team sizes between 3 and 5 (Table II). When the only requirements were Python and Design coverage, the solver found valid teams after 19 node visits and 6 backtracks in 0.11 ms, with team sizes of 5, 4 and 3 (size variance 0.667). Adding a must-be-together pair (Alice and Dave) and a must-not-be-together pair (Bob and Carol) was also solved without relaxation, in 18 nodes.

Requiring a Team Lead in every team was a harder constraint, since only Alice, Dave and Liam qualify. The

solver still satisfied it by placing one of them in each team, but needed 875 node visits and 938 backtracks (3.53 ms), showing how scarce eligible roles increase the search effort.

TABLE II. SOLVER BEHAVIOR ON FIVE SCENARIOS

Scenario (critical requirements)	Nodes	Back-tracks	Time (ms)	Outcome
S1: Python, Design	19	6	0.11	All met
S2: S1 + together/apart pairs	18	6	0.10	All met
S3: S1 + Team Lead	875	938	3.53	All met
S4: S3 + Project Manager	19*	6*	85.54	Relaxed
S5: Python, Design, ML, Testing	19*	6*	80.40	Relaxed

*Counts of the second (relaxed) pass. Time includes the first pass, which stopped at the 20,000-node limit.

C. Constraint Relaxation and Diagnostics

Scenarios S4 and S5 are over-constrained. In S4 only Dave can serve as Project Manager, so at most one of the three teams can contain one. The solver relaxed the requirement, returned valid teams, and reported that Team 2 lacked a Team Lead and a Project Manager and Team 3 lacked a Project Manager. In S5 only Ivan and Judy have Testing skill, so one team could not be covered, and the tool reported Testing as missing for Team 1. In both cases the solver returned usable teams together with an explicit list of unmet requirements.

D. Fairness and Meeting-Point Search

For the teams of scenario S1, A* and Dijkstra returned identical optimal distances, but A* expanded fewer nodes (Table III). Across all 132 ordered pairs of campus locations, A* averaged 3.66 node expansions per query compared with 7.14 for Dijkstra, with no difference in the distances found.

TABLE III. MEETING POINTS (TOTAL-DISTANCE CRITERION; MAP UNITS)

Team	Size	Meeting point	Total	Max walk	A* nodes	Dijkstra nodes
1	5	Main Gate	19.41	5.66	195	391
2	4	Sports Complex	16.91	11.84	171	308
3	3	Main Gate	10.33	5.85	117	244
All	12	—	—	—	483	943

Overall, A* used about 49% fewer node expansions (483 versus 943). The choice of criterion also matters. For Team 2, minimizing the longest walk selected Main Gate instead of Sports Complex, reducing the longest individual walk from 11.84 to 8.23 units (about 31% shorter) at the price of a longer total distance (16.91 to 23.23 units, about 37% more). Teams 1 and 3 had the same meeting point under both criteria.

E. Scalability on Synthetic Cohorts

To check behavior beyond the sample cohort, ten random cohorts were generated for each size, where each participant had one to three skills from a pool of eight, and Python and Design coverage was required in every team (Table IV).

TABLE IV. RESULTS ON RANDOM COHORTS (10 PER SIZE)

People	Teams	Solved	Relaxed	Avg. nodes	Avg. time (ms)
12	3	10/10	7	19.0	62.3
20	4	10/10	6	132.2	92.6
30	5	10/10	7	2700.6	88.0
40	6	10/10	3	66.5	32.8
60	8	10/10	2	2211.7	24.2

All 50 cohorts produced valid teams. Because skills were assigned at random, many cohorts could not satisfy full coverage and were relaxed. The average time is therefore dominated by runs in which the first pass exhausted the node limit before relaxation, which explains why larger cohorts were not slower.

F. Overall Discussion

The proposed system demonstrates the potential of classical artificial intelligence for integrating role reasoning, team formation, fairness evaluation, and logistics into a single platform. Instead of treating these as independent tasks, the AI Team Formation Tool connects them: roles inferred by the logic engine become constraints for the CSP solver, and the formed teams become the input of the meeting-point search.

The major advantages are transparency and graceful handling of conflicts. Every inferred role has a reasoning trace, and unmet requirements are reported explicitly. However, the evaluation has limitations. It used a small sample cohort and synthetic data rather than a real class, no user study was conducted, and the rule thresholds are illustrative and should be set by the organizer. The 20,000-node limit means that a relaxed result may reflect an exhausted search budget rather than proven infeasibility. The balance metrics are descriptive and are not optimized directly, and the query interface for target roles is a lightweight check rather than a complete proof-tree generator.

Future versions can incorporate full backward-chaining proof trees, a skill-gap repair step that swaps members between teams, uninformed-search baselines (BFS, DFS and uniform-cost search) for comparison, optimization of balance objectives, soft preferences, and evaluation with real classroom data and user feedback.

This paper presented the **AI Team Formation Tool**, an intelligent system designed to address the limitations of random and manual team formation. The system considers participant skills, attendance, portfolio status, leadership experience, and location to infer eligible roles, form teams under hard constraints, report unmet requirements, evaluate fairness, and recommend meeting points. By combining forward chaining, constraint satisfaction, and A* search, the tool provides a unified and explainable approach to team formation.

The system is intended to serve as a supportive decision-support platform rather than a replacement for the judgment of instructors and organizers. Overall, the AI Team Formation Tool demonstrates the potential of artificial intelligence to provide fair, transparent, and practical team allocation.

AUTHOR CONTRIBUTION

Deepashri Murali contributed to problem identification, system design, development of the rule engine and CSP solver, experimental evaluation, and manuscript writing.
Bushra Banu KM contributed to the literature survey, development of the meeting-point search and user interface, testing, result analysis, and documentation.
Both authors contributed to the development of the proposed system, reviewed the research work, and approved the final manuscript.